

KFS — Use Case Data Flows

Reference doc tracing **what happens underneath**, not what a screen looks like: for each scenario, which Controller → Service → Repository calls fire, in what order, and which tables end up with new or changed rows.

Grounded directly in `kfs-schema-create.sql` and the current package-by-feature implementation (ADR-0006). Covers the six core happy-path flows; validation/rejection paths and edge cases are intentionally out of scope for this pass.

Reading the tables below

Column	Meaning
Step	Order of execution
Component	The class/method actually doing the work
Action	What it does
Table(s) Touched	Which table(s) get a row inserted/updated
Data Written	The columns that matter for this flow

Use Case 1: New User's First KnowledgeBase (INBOX auto-provisioning)

Trigger: A user needs a place for entries to land before they're organized — every user gets exactly one default/INBOX KnowledgeBase (enforced: never zero, never two).

Narrative: `KnowledgeBaseController.createDefault` receives an `ownerId` and calls `KnowledgeBaseService.createDefaultKnowledgeBase`. The service first checks whether this owner already has a default KB via `KnowledgeBaseRepository.findByOwnerIdAndIsDefaultTrue` — if one exists, it throws `IllegalStateException` rather than silently creating a second one. Otherwise it builds a new `KnowledgeBase` named "Inbox", `isDefault = true`, `ownerId = createdBy = ownerId`, and saves it. One row lands in `knowledge_base`.

Step	Component	Action	Table(s) Touched	Data Written
1	KnowledgeBaseController.createDefault	Receives ownerId	—	—
2	KnowledgeBaseServiceImpl.createDefaultKnowledgeBase	Checks for existing default	knowledge_base (read)	—
3	KnowledgeBaseServiceImpl.createDefaultKnowledgeBase	Builds + saves new KB	knowledge_base (insert)	name='Inbox', owner_id, is_default=TRUE, created_by=owner_id

Use Case 2: Creating an Entry

Trigger: A user captures a new piece of knowledge. It always lands in a KnowledgeBase with `status_id = INBOX` — it is not yet filed under any Node.

Narrative: The intended flow is `EntryController` → `EntryService.create(EntryDto)` → `EntryRepository.save`, producing one row in `entry` with `source_id` and `content_type_id` required (both `NOT NULL`), `status_id` pointing at the seeded `INBOX` row in `entry_status`, and `knowledge_base_id` pointing at an existing KB (FK-enforced).

Status note: `EntryService.create(EntryDto)` doesn't exist yet — `EntryService` currently only exposes `findById`/`findByKnowledgeBaseId`. This is the intended design once that method lands (it's the blocker noted in `EntryFilingIntegrationTest`), not a flow that's live today.

Step	Component	Action	Table(s) Touched	Data Written
1	<code>EntryController</code> <i>(planned)</i>	Receives <code>EntryDto</code>	—	—
2	<code>EntryService.create</code> <i>(planned)</i>	Validates + builds <code>Entry</code>	—	—
3	<code>EntryRepository.save</code>	Persists	<code>entry</code> (insert)	<code>knowledge_base_id</code> , <code>status_id=INBOX</code> , <code>source_id</code> , <code>content_type_id</code> , <code>created_by</code>

Use Case 3: Filing an Entry into a Node

Trigger: A user organizes an Entry that's sitting in INBOX by placing it under a specific Node in the tree. This is the moment an Entry stops being "unfiled" — filed status is *derived* from the presence of an `entry_node` row, not a flag on `entry` itself.

Narrative: `NodeController.fileEntry` calls `NodeService.fileEntry(nodeId, entryId)`. The service confirms the Node exists (`ResourceNotFoundException` if not), counts how many Entries are already filed under that Node via `EntryNodeRepository.findById_NodeIdOrderByDisplayOrder` to compute the next `display_order`, then saves a new `EntryNode` row keyed on the composite `(entry_id, node_id)`. No `entry` row is touched — filing is purely additive, in the `entry_node` association table.

Step	Component	Action	Table(s) Touched	Data Written
1	<code>NodeController.fileEntry</code>	Receives <code>nodeId</code> , <code>entryId</code>	—	—
2	<code>NodeServiceImpl.fileEntry</code>	Confirms Node exists	<code>node</code> (read)	—
3	<code>NodeServiceImpl.fileEntry</code>	Computes next <code>display_order</code>	<code>entry_node</code> (read)	—
4	<code>EntryNodeRepository.save</code>	Persists the filing	<code>entry_node</code> (insert)	<code>entry_id</code> , <code>node_id</code> , <code>display_order</code> , <code>created_by</code>

Known gap: the schema comment on `entry_node` calls out a rule that "`entry.knowledge_base_id` must match `node.knowledge_base_id` for any linked Node" and explicitly notes it's *not* enforceable in plain SQL — it has to be checked in the service layer. `NodeServiceImpl.fileEntry` doesn't check this yet, so today it's technically possible to file an Entry from one KnowledgeBase into a Node that belongs to a different one. Worth a follow-up fix.

Use Case 4: Tagging an Entry

Trigger: A user attaches an existing, KB-scoped Tag to an Entry.

Narrative: Prerequisite: the Tag already exists (created separately via `TagService.create`, which enforces `unique(knowledge_base_id, name)` at the service layer before the DB constraint would). `TagController.tagEntry` calls `TagService.tagEntry(entryId, tagId, userId)`. The service confirms the Tag exists, then checks whether this exact `(entry_id, tag_id)` pairing already exists via `EntryTagRepository.findById_EntryIdAndId_TagId` — tagging is idempotent, so a second call with the same pair is a silent no-op rather than an error. If it's new, it saves an `EntryTag` row with `added_at = now()` and `created_by = userId`.

Step	Component	Action	Table(s) Touched	Data Written
1	<code>TagController.tagEntry</code>	Receives <code>tagId</code> , <code>entryId</code> , <code>userId</code>	—	—
2	<code>TagServiceImpl.tagEntry</code>	Confirms Tag exists	<code>tag</code> (read)	—
3	<code>TagServiceImpl.tagEntry</code>	Checks for existing pairing	<code>entry_tag</code> (read)	—
4	<code>EntryTagRepository.save</code>	Persists the tagging (if new)	<code>entry_tag</code> (insert)	<code>entry_id</code> , <code>tag_id</code> , <code>added_at</code> , <code>created_by</code>

Use Case 5: Creating a Relationship Between Two Entries

Trigger: A user connects two existing Entries in the knowledge graph — independent of where either one sits in the Node tree, and possibly across KnowledgeBase boundaries.

Narrative: `RelationshipController.create` calls `RelationshipService.create(dto)`. The service first rejects self-referencing relationships (`sourceEntryId == targetEntryId`) with `IllegalArgumentException` before touching the database. It then validates *both* ends exist by calling `EntryService.findById` for source and target — this is a cross-package call to the public `EntryService` seam, not a direct query against the entry table, keeping the relationship package from reaching around `entry`'s public interface. If either lookup throws `ResourceNotFoundException`, creation stops there. Otherwise it builds and saves a `Relationship` row.

Step	Component	Action	Table(s) Touched	Data Written
1	<code>RelationshipController.create</code>	Receives <code>RelationshipDto</code>	—	—
2	<code>RelationshipServiceImpl.create</code>	Rejects self- reference	—	—
3	<code>RelationshipServiceImpl.create</code> → <code>EntryService.findById</code> (×2)	Validates both ends exist	<code>entry</code> (read ×2)	—
4	<code>RelationshipRepository.save</code>	Persists	<code>relationship</code> (insert)	<code>source_entry_id</code> , <code>target_entry_id</code> , <code>relationship_type_id</code> , <code>notes</code> , <code>created_by</code>

Known gap: the schema also has `uq_rel_edge` — a unique constraint on `(source_entry_id, target_entry_id, relationship_type_id)` — but the service doesn't pre-check for that duplicate the way `TagServiceImpl.create` does for tag names. A repeat relationship would currently surface as a raw `DataIntegrityViolationException` from the DB rather than a clean `DuplicateResourceException`. Same category of fix as the Node/KnowledgeBase gap above, just not applied here yet.

Use Case 6: Creating a Secondary (Non-Default) KnowledgeBase

Trigger: A user who already has their default INBOX wants a separate, purpose-built KnowledgeBase — e.g. splitting "Personal" from "Work."

Narrative: `KnowledgeBaseController.create` calls `KnowledgeBaseService.create(dto)`. Unlike Use Case 1, there's no default-uniqueness check here — the service unconditionally sets `isDefault = false` regardless of what the incoming DTO says, so a regular KB can never accidentally become a second default. One row lands in `knowledge_base`.

Step	Component	Action	Table(s) Touched	Data Written
1	<code>KnowledgeBaseController.create</code>	Receives <code>KnowledgeBaseDto</code>	—	—
2	<code>KnowledgeBaseServiceImpl.create</code>	Forces <code>is_default = false</code>	—	—
3	<code>KnowledgeBaseRepository.save</code>	Persists	<code>knowledge_base</code> (insert)	<code>name</code> , <code>owner_id</code> , <code>is_default=FALSE</code> , <code>created_by=owner_id</code>

Summary: Who Can Write to Which Table

Table	Written by
<code>knowledge_base</code>	<code>KnowledgeBaseService</code> only
<code>entry</code>	<code>EntryService</code> only (once <code>create</code> exists)
<code>node</code>	<code>NodeService</code> only
<code>entry_node</code>	<code>NodeService</code> only
<code>tag</code>	<code>TagService</code> only
<code>entry_tag</code>	<code>TagService</code> only
<code>relationship</code>	<code>RelationshipService</code> only — reads <code>entry</code> via <code>EntryService</code> , never writes it

This one-writer-per-table pattern is the practical expression of ADR-0006's package-by-feature boundaries: every table has exactly one service package authorized to mutate it, and cross-package reads always go through the other package's public service interface, never its repository or entity.